

# **AI ASSISTED CODING**

## **ASSIGNMENT 17.2**

**NAME:** KADARLA HASINI

**HT NO:** 2403A52062

**BATCH NO:** 01

### **TASK 1~**

Clean raw social media posts dataset.

Instructions:

- Remove stopwords, punctuation, and special symbols from post text.
- Handle missing values in likes and shares columns.
- Convert timestamp to datetime and extract features (hour, weekday).
- Detect and remove spam/duplicate posts.

## PROMPT:

Write HTML, CSS, and JavaScript code that uploads and preprocesses a raw social media posts dataset (CSV file).

The code should:

- Remove stopwords, punctuation, and special characters from the post text.
- Handle missing values in the likes and shares columns (fill with median).
- Convert timestamps to datetime format and extract useful features like posting hour and weekday.
- Detect and remove duplicate or spam posts automatically.
- Display a short before-and-after summary (e.g., total posts before/after cleaning).
- Allow the user to download the cleaned dataset as a new CSV file.

## CODE:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <title>Clean Social Media Posts</title>
6  <script src="https://cdn.jsdelivr.net/npm/papaparse@5.3.2/papaparse.min.js"></script>
7  </head>
8  <body>
9  <h2>Social Media Post Cleaner</h2>
10 <input type="file" id="file"><button onclick="cleanData()">Clean & Download</button>
11 <p id="status"></p>
12
13 <script>
14 function cleanData() {
15     const file = document.getElementById('file').files[0];
16     if (!file) return alert("Upload a CSV first!");
17     document.getElementById("status").textContent = "Processing...";
18
19     Papa.parse(file, {
20         header: true,
21         complete: function(results) {
22             let data = results.data;
23
24             // --- Utility helpers ---
25             const stopwords = new Set(["a", "an", "the", "and", "is", "in", "on", "of", "for", "to", "it", "wit
26             const cleanText = t => t.toLowerCase()
27                 .replace(/^[^w\s]/g, '') // remove pu (method) Set<string>.has(value: string): boolean
28                 .split(/\s+/).filter(w => !stopwords.has(w)).join(' ');
29
30             const isSpam = t => /(buy now|click here|free|subscribe|http|www)/i.test(t);
31
32             const median = arr => {
33                 arr = arr.filter(x=>x!="" && !isNaN(x)).map(Number).sort((a,b)=>a-b);
34                 return arr.length ? arr[Math.floor(arr.length/2)] : 0;
35             };
36
```

```

37 // --- Detect columns ---
38 const cols = Object.keys(data[0]);
39 const textCol = cols.find(c=>/text|post|content/i.test(c)) || cols[0];
40 const timeCol = cols.find(c=>/time|date/i.test(c));
41 const likesCol = cols.find(c=>/like/i.test(c));
42 const shareCol = cols.find(c=>/share|retweet/i.test(c));
43
44 const likesMed = median(data.map(r=>r[likesCol]));
45 const shareMed = median(data.map(r=>r[shareCol]));
46
47 let seen = new Set();
48 let cleaned = [];
49
50 data.forEach(r => {
51   let txt = cleanText(r[textCol] || "");
52   if (!txt || isSpam(txt)) return;
53   if (seen.has(txt)) return;
54   seen.add(txt);
55
56   // Handle missing likes/shares
57   if (likesCol) r[likesCol] = r[likesCol] || likesMed;
58   if (shareCol) r[shareCol] = r[shareCol] || shareMed;
59
60   // Timestamp → datetime, hour, weekday
61   if (timeCol) {
62     const d = new Date(r[timeCol]);
63     if (!isNaN(d)) {
64       r.hour = d.getHours();
65       r.weekday = d.toLocaleString('en', { weekday: 'long' });
66     }
67   }
68 }

```

```

68
69     r.cleaned_text = txt;
70     cleaned.push(r);
71 });
72
73 // Download cleaned CSV
74 const csv = Papa.unparse(cleaned);
75 const blob = new Blob([csv], {type:"text/csv"});
76 const a = document.createElement("a");
77 a.href = URL.createObjectURL(blob);
78 a.download = "cleaned_posts.csv";
79 a.click();
80
81 document.getElementById("status").textContent = `Done! Cleaned rows: ${cleaned.length}`;
82 }
83 });
84 }
85 </script>
86 </body>
87 </html>
88

```

## OUTPUT:

### Social Media Post Cleaner

Choose File No file chosen

Clean & Download

## CODE EXPLANATION:

This HTML tool uploads a CSV of social media posts, cleans the text by removing stopwords and punctuation, fills missing likes/shares with median values, converts timestamps to extract hour and weekday, detects and removes spam or duplicate posts, and then downloads the cleaned data as a new CSV file.

## TASK 2~

Preprocess a stock market dataset.

Instructions:

- Handle missing values in closing\_price and volume.
- Create lag features (1-day, 7-day returns).
- Normalize volume column using log-scaling.
- Detect outliers in closing\_price using IQR method

### PROMPT:

Write HTML, CSS, and JavaScript code to upload and preprocess a stock market dataset (CSV file). The code should:

- Handle missing values in closing\_price and volume columns (fill with mean or median).
- Create lag features for 1-day and 7-day returns.
- Normalize the volume column using log-scaling.
- Detect outliers in closing\_price using the Interquartile Range (IQR) method.
- Display a brief summary (number of rows before and after cleaning, detected outliers).

- Allow the user to download the cleaned dataset as a new CSV file

## CODE:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <title>Stock Data Preprocessing</title>
6  <script src="https://cdn.jsdelivr.net/npm/papaparse@5.3.2/papaparse.min.js"></script>
7  </head>
8  <body>
9  <h3>Stock Market Preprocessor</h3>
10 <input type="file" id="file"><button onclick="process()">Process & Download</button>
11 <p id="status"></p>
12
13 <script>
14 function process() {
15   const f = document.getElementById('file').files[0];
16   if (!f) return alert("Upload a CSV first!");
17   Papa.parse(f, {
18     header: true,
19     complete: r => {
20       let d = r.data;
21
22       // Fill missing values
23       const mean = col => d.map(x=>+x[col]).filter(v=>!isNaN(v)).reduce((a,b)=>a+b,0)/d.length
24       const med = col => {let v=d.map(x=>+x[col]).filter(v=>!isNaN(v)).sort((a,b)=>a-b);return
25       let mClose = mean('closing_price'), mVol = med('volume');
26       d.forEach(x=>{
27         x.closing_price = +x.closing_price || mClose;
28         x.volume = +x.volume || mVol;
29       });
30
31       // Lag returns
32       for(let i=1;i<d.length;i++){
33         d[i].ret_1d = ((d[i].closing_price - d[i-1].closing_price)/d[i-1].closing_price).toFixed
34         if(i>=7) d[i].ret_7d = ((d[i].closing_price - d[i-7].closing_price)/d[i-7].closing_pri
35       }
```

```

36
37 // Log scale volume
38 d.forEach(x=>x.vol_log = Math.log1p(x.volume).toFixed(4));
39
40 // Outlier detection (IQR)
41 let prices = d.map(x=>x.closing_price);
42 let Q1 = quantile(prices,0.25), Q3 = quantile(prices,0.75), IQR=Q3-Q1;
43 d.forEach(x=>x.is_outlier = (x.closing_price<Q1-1.5*IQR || x.closing_price>Q3+1.5*IQR));
44
45 // Download cleaned CSV
46 const csv = Papa.unparse(d);
47 const a = document.createElement("a");
48 a.href = URL.createObjectURL(new Blob([csv], {type:"text/csv"}));
49 a.download = "cleaned_stock.csv"; a.click();
50 document.getElementById('status').textContent = "Done! Cleaned file downloaded.";
51 }
52 });
53 }
54
55 function quantile(arr,q){
56   arr=arr.slice().sort((a,b)=>a-b);
57   const pos=(arr.length-1)*q, base=Math.floor(pos), rest=pos-base;
58   return arr[base]+(arr[base+1]-arr[base])*rest;
59 }
60 </script>
61 </body>
62 </html>
63

```

## OUTPUT:

### Stock Market Preprocessor

Choose File No file chosen

Process & Download

## CODE EXPLANATION:

This HTML and JavaScript code uploads a stock market CSV, fills missing closing prices with the mean and volume



with the median, calculates 1-day and 7-day returns, applies log normalization to the volume column, detects outliers in closing price using the IQR method, and downloads the cleaned dataset.

### **TASK 3~**

Clean and preprocess IoT temperature and humidity logs.

Instructions:

- Handle missing values using forward fill.
- Remove sensor drift (apply rolling mean).
- Normalize readings using standard scaling.
- Encode categorical sensor IDs

### **PROMPT:**

Write HTML, CSS, and JavaScript code that uploads and cleans IoT temperature and humidity log data from a CSV file. The code should:

- Handle missing values using forward fill.

- Remove sensor drift by applying a rolling mean (smoothing).
- Normalize temperature and humidity readings using standard (z-score) scaling.
- Encode categorical sensor IDs into numeric codes.
- Display a short before-and-after summary (missing values, average readings).
- Allow the user to download the cleaned and processed dataset as a new CSV file.

## CODE:

```
desktop / AI-assisted coding / ... / Untitled 2.html / ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <title>IoT Data Preprocessor</title>
6  <script src="https://cdn.jsdelivr.net/npm/papaparse@5.3.2/papaparse.min.js"></script>
7  </head>
8  <body>
9  <h3>IoT Temperature & Humidity Preprocessor</h3>
10 <input type="file" id="file"><button onclick="process()">Process & Download</button>
11 <p id="status"></p>
12
13 <script>
14 function process() {
15   const f = document.getElementById('file').files[0];
16   if (!f) return alert("Upload a CSV first!");
17   Papa.parse(f, {
18     header: true,
19     complete: r => {
20       let d = r.data;
21
22       // 1 Forward fill missing values
23       ['temperature', 'humidity'].forEach(col=>{
24         for(let i=1;i<d.length;i++)
25           if(!d[i][col]) d[i][col]=d[i-1][col];
26       });
27
28       // 2 Remove sensor drift (rolling mean, window=3)
29       const smooth = (arr) => arr.map((_,i)=>
30         (arr.slice(Math.max(0,i-2),i+1).reduce((a,b)=>a+Number(b||0),0))/
31         arr.slice(Math.max(0,i-2),i+1).length);
32       let temps = d.map(x=>+x.temperature);
33       let hums = d.map(x=>+x.humidity);
34       let tSmooth = smooth(temps), hSmooth = smooth(hums);
35       d.forEach((x,i)=>{x.temperature=tSmooth[i];x.humidity=hSmooth[i]});
36     }
37   });
38 }
```

```

36
37 // 3 Standard scaling
38 const scale = (arr) => {
39   let m = arr.reduce((a,b)=>a+b,0)/arr.length;
40   let s = Math.sqrt(arr.reduce((a,b)=>a+(b-m)**2,0)/arr.length);
41   return arr.map(v=>((v-m)/s).toFixed(4));
42 };
43 let tScaled = scale(d.map(x=>x.temperature));
44 let hScaled = scale(d.map(x=>x.humidity));
45 d.forEach((x,i)=>{x.temp_scaled=tScaled[i];x.hum_scaled=hScaled[i]});
46
47 // 4 Encode sensor IDs
48 let ids=[...new Set(d.map(x=>x.sensor_id))];
49 d.forEach(x=>x.sensor_code=ids.indexOf(x.sensor_id));
50
51 // Download cleaned CSV
52 const csv = Papa.unparse(d);
53 const a=document.createElement("a");
54 a.href=URL.createObjectURL(new Blob([csv],{type:"text/csv"}));
55 a.download="cleaned_iot.csv"; a.click();
56 document.getElementById('status').textContent="Done! Cleaned file downloaded.";
57 }
58 });
59 }
60 </script>
61 </body>
62 </html>
63

```

## OUTPUT:

### IoT Temperature & Humidity Preprocessor

Choose File No file chosen

Process & Download

## CODE EXPLANATION:

This HTML and JavaScript code uploads an IoT CSV, fills missing temperature and humidity values using forward fill, smooths readings to remove sensor drift with a rolling mean, standardizes them via z-score scaling, encodes

sensor IDs into numeric labels, and downloads the fully cleaned and normalized dataset.

## **TASK 4~**

A streaming platform wants to analyze customer reviews.

Instructions:

- Standardize text (lowercase, remove HTML tags).
- Tokenize and encode reviews using AI-assisted methods (TF-IDF or embeddings).
- Handle missing ratings (fill with median).
- Normalize ratings (0–10 → 0–1 scale).
- Generate a before vs after summary report.

## **PROMPT:**

Write HTML, CSS, and JavaScript code to preprocess customer review data from a streaming platform. The code should:

- Convert all text to lowercase and remove HTML tags.

- Tokenize and encode reviews using TF-IDF.
- Handle missing ratings by filling with the median.
- Normalize ratings (0–10 → 0–1 scale).
- Generate a before vs after summary report showing text cleaning, rating normalization, and encoding results.

The output should be a cleaned dataset ready for sentiment classification and available for download as a CSV file.

## CODE:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <title>Customer Review Preprocessor</title>
6  <script src="https://cdn.jsdelivr.net/npm/papaparse@5.3.2/papaparse.min.js"></script>
7  <script src="https://cdn.jsdelivr.net/npm/tfidf.js@1.0.1/dist/tfidf.min.js"></script>
8  </head>
9  <body>
10 <h3> 📄 Streaming Platform Review Preprocessor</h3>
11 <input type="file" id="file"><button onclick="process()">Process & Download</button>
12 <pre id="report"></pre>
13
14 <script>
15 function process(){
16   const f=document.getElementById('file').files[0];
17   if(!f) return alert("Upload a CSV first!");
18   Papa.parse(f,{
19     header:true,
20     complete:r=>{
21       let d=r.data;
22
23       // 1 Text cleaning
24       d.forEach(x=>{
25         x.review=(x.review||'').toLowerCase().replace(/<[^>]+>/g,'');
26       });
27
28       // 2 Handle missing ratings (median)
29       let valid=d.map(x=>+x.rating).filter(v=>!isNaN(v));
30       let med=valid.sort((a,b)=>a-b)[Math.floor(valid.length/2)];
31       d.forEach(x=>x.rating=x.rating?+x.rating:med);
32
33       // 3 Normalize ratings (0-1)
34       d.forEach(x=>x.rating_norm=(x.rating/10).toFixed(2));
35     }
36   });
37 }
```

```

35
36 // 4 Simple TF-IDF token encoding
37 let tfidf=new TFIDF();
38 d.forEach(x=>tfidf.addDocument(x.review));
39 d.forEach((x,i)=>x.vector=tfidf.listTerms(i).slice(0,5).map(v=>v.term).join(' '));
40
41 // 5 Before vs after summary
42 let rep={}
43   "Total Reviews":d.length,
44   "Median Rating":med,|
45   "Avg Length":(d.reduce((a,b)=>a+b.review.length,0)/d.length).toFixed(1),
46   "Example Encoded":d[0].vector
47 };
48 document.getElementById("report").textContent=JSON.stringify(rep,null,2);
49
50 // 6 Download cleaned CSV
51 const csv=Papa.unparse(d);
52 const a=document.createElement("a");
53 a.href=URL.createObjectURL(new Blob([csv],{type:"text/csv"}));
54 a.download="cleaned_reviews.csv"; a.click();
55 }
56 });
57 }
58 </script>
59 </body>
60 </html>
61

```

## OUTPUT:



### Streaming Platform Review Preprocessor

Choose File No file chosen

Process & Download

## CODE EXPLANATION:

This HTML and JavaScript code uploads a customer review CSV, cleans review text by converting to lowercase and removing HTML tags, fills missing ratings with the median, normalizes ratings to a 0–1 scale, encodes



reviews using a TF-IDF method, generates a before–after summary, and downloads the cleaned dataset.